Clase 10 · El socket servidor paso a paso

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 29 sep 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

"En dos semanas vamos a tener la evaluación parcial… tienen que crear un cliente-servidor." Documentación de referencia: The Open Group (POSIX).

1. socket()

`int socket(int domain, int type, int protocol);` (en `<sys/socket.h>`): crea un extremo **no ligado** de comunicación.

Argumento	Valores
domain	**AF_INET** (IPv4); también AF_INET6, AF_UNIX/AF_LOCAL, AF_BLUETOOTH, AF_PACKET
type	**SOCK_STREAM** (TCP, confiable) o **SOCK_DGRAM** (UDP); SOCK_SEQPACKET, SOCK_RAW
protocol	**0** = el predeterminado (TCP para AF_INET + SOCK_STREAM) o IPPROTO_TCP

Devuelve un **descriptor no negativo** (normalmente **3**, porque 0, 1 y 2 son stdin, stdout y stderr) o **−1** con el código en `errno`.

2. Inicializar struct sockaddr_in

```
memset(&servidor, 0, sizeof(servidor));     /* quitar basura */
servidor.sin_family = AF_INET;              /* igual al dominio de socket() */
servidor.sin_addr.s_addr = INADDR_ANY;      /* doble punto: sin_addr es struct in_addr
*/
servidor.sin_port = htons(4898);            /* Host TO Network Short */
memset(servidor.sin_zero, '\0', 8);         /* relleno histórico */
```

¿Por qué el servidor pone una IP?

Un equipo tiene varias interfaces (Ethernet, Wi-Fi, loopback, virtuales). **INADDR_ANY** escucha por **todas** sus IP: un cliente local entra por 127.0.0.1 y uno externo por la IP de la tarjeta.

- **htons:** H = host, TO, N = network, S = short (16 bits, adecuado para puertos): convierte al orden de bytes de red.
- `sin_zero` iguala el tamaño con `struct sockaddr` genérica (opcional en Linux moderno).

3. bind()

`int bind(int socket, const struct sockaddr *address, socklen_t address_len);` — **liga** el descriptor con IP y puerto.

```
if (bind(fd_s, (struct sockaddr *) &servidor, sizeof(servidor)) == -1) {
    perror("bind"); exit(EXIT_FAILURE);
}
```

Puertos **< 1024** requieren root/sudo; un usuario normal usa 1024–65535 si están libres. Requiere el **cast** a `(struct sockaddr *)`.

4. listen()

`int listen(int socket, int backlog);` — modo de escucha activa; **backlog** = máximo de conexiones **pendientes en cola** (no el total de clientes atendidos). Detrás está el **saludo de tres vías** (SYN, SYN-ACK, ACK).

5. accept() y los dos sockets

`int accept(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len);` — **bloqueante**; el kernel **rellena** la estructura del cliente; el tercer argumento es un **apuntador** (`&longitud_cliente`).

	Socket de conexión	Socket de comunicación
Se crea con	`socket()`	Valor de retorno de `accept()`
Variable	`fd_s`	`fd_c`
Valor típico	3	4
Función	Guarda IP y puerto de escucha; recibe y encola solicitudes	Guarda IP y puerto del cliente; flujo de datos (read/write, send/recv)

> **La base del curso**
>
> Todo servidor maneja **al menos dos sockets**: `fd_s` acepta, `fd_c` comunica.
>
> **Pregunta:** ¿un cliente puede conectarse a varios servidores a la vez? Sí (servidor web 443, MySQL 3306, SMTP 25): una `sockaddr_in` y un socket por servicio.

6. Los dos close() del servidor

- **Cliente:** `close(fd)` una vez.
- **Servidor secuencial:** atiende a uno a la vez; al terminar ese cliente hace `close(fd_c)` y regresa en bucle a `accept()`. Solo al apagarse hace `close(fd_s)`.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Secuencia del servidor (examen)

`socket()` (AF_INET, SOCK_STREAM, 0 → fd 3) → inicializar `sockaddr_in` (memset, AF_INET, `sin_addr.s_addr = INADDR_ANY`, `htons(puerto)`) → `bind()` (cast y sizeof) → `listen(fd, backlog)` → `accept()` (bloqueante, devuelve fd_c = 4).

Concepto	Para recordar
fd_s vs. fd_c	Conexión vs. comunicación
INADDR_ANY	Escuchar en todas las interfaces
htons	Puerto a orden de red (16 bits)
Puertos < 1024	Requieren root
backlog	Cola de pendientes
close()	Servidor: fd_c por cliente, fd_s al apagar